UNIT 3 – TREE DATA STRUCTURES

Introduction – Types of Trees – Creating a Binary Tree from a General Tree – Traversing a Binary Tree – Binary Search Trees – Operations on a Binary Search Trees – Threaded Binary Trees – AVL Trees– Binary Heaps – Applications of Heaps.

INTRODUCTION

What is a Tree?

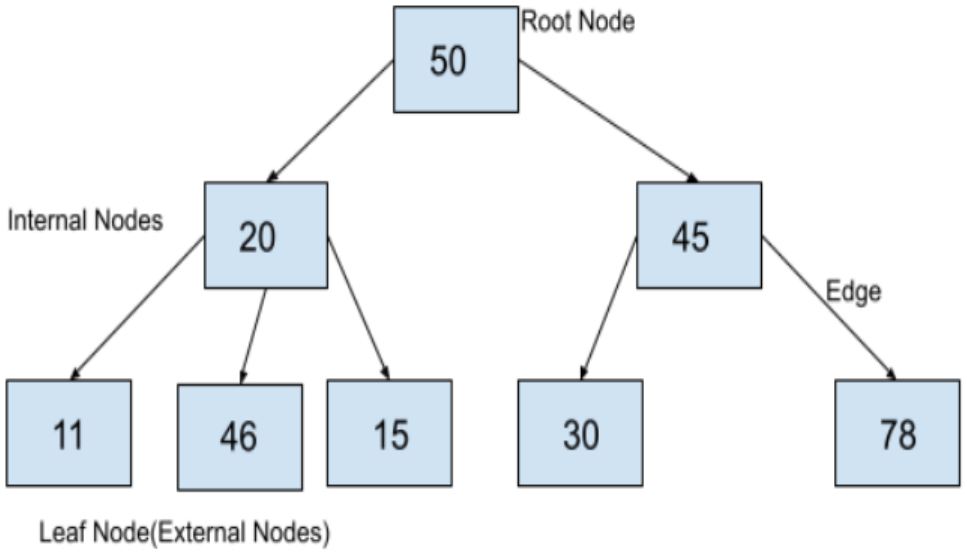
- A **tree** is a **non-linear** data structure made up of **nodes** connected by **edges**. It starts with a **root node**, and each node can have **children**. A common type of tree is the **binary tree**, where each node has at most **two children**.

Why Use Trees?

- Efficient for searching and sorting (like Binary Search Tree)
- Represent hierarchical data (e.g., file systems, organization charts)
- Backbone for complex data structures (heaps, tries, syntax trees)

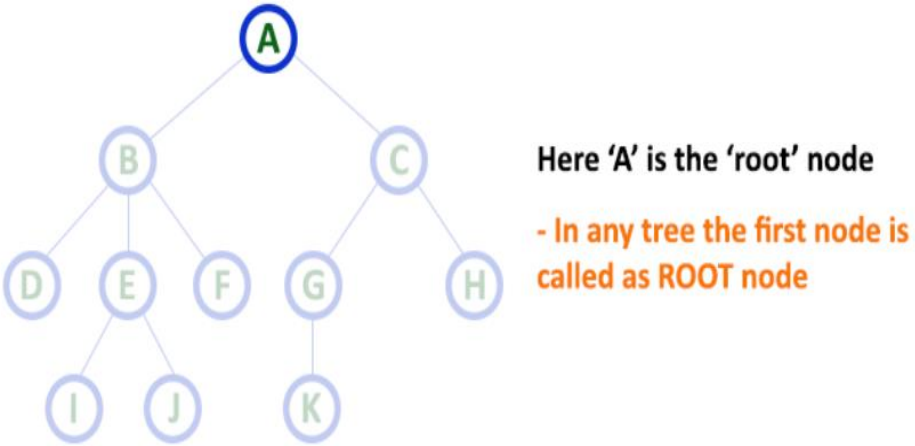
Tree ADT

- A Tree is a Data structure in which data items are connected using references in a hierarchical manner.
- **Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.**
- Each Tree consists of a root node from which can access each element of the tree.
- Starting from the root node, each node contains zero or more nodes conr

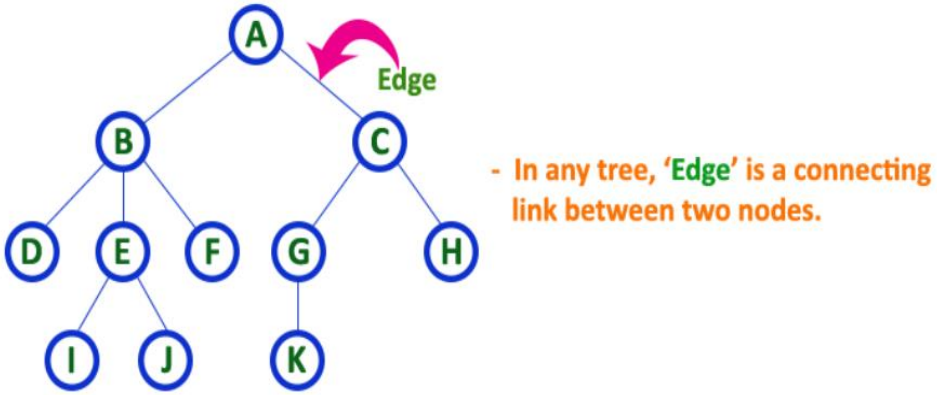


Tree Terminologies

1. **Root node:** In a tree data structure, the first node is called as **Root Node**. Every tree must have a root node. The root node is the origin of the tree data structure. In

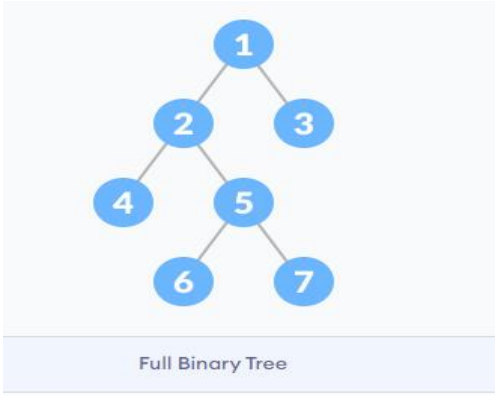


2. Edge: In a tree data structure, the connecting link between any two nodes is called as **EDGE**. In a tree with '**N**' number of nodes there will be a maximum of '**N-1**' number of edges.



Types of Binary tree

Full Binary Tree: A full Binary tree is a special type of binary tree in which every parent node/internal node has either two or no children.



Perfect Binary Tree: A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level**.**

